Patch-Effect Signal Compression: Localizing Activation-Patching Information in Signed Edge-Slot Identities

Ruben Fernández-Boullón David N. Olivieri*

May 4, 2026

Abstract

Mechanistic interpretability exposes rich node-level signals via activation patching, but the resulting tensors are difficult to compare and scale. We study how much of this patch-effect signal is preserved when compressing patch-effect profiles through different representation schemes. Using slice classification as a signal-preservation probe—not as an end in itself—we systematically compress raw patch-effect tensors into structured graph features on two mechanistically distinct tasks with GPT-2: the indirect object identification (IOI) task (name-binding circuit) and a controlled induction-head task (token-repetition circuit). Across both tasks, we find that the discriminative patch-effect signal is localised in signed edge-slot identities: on IOI, fixed-layout signed features achieve 0.9896 mean accuracy but require $N(N-1)$ coordinates. Hashed fixed-layout features reach 0.9375 accuracy with 1024 dimensions, recovering 94.7% of the fixed-layout signal at 7.2% of the dimensionality. Coarse bucket counts reach 0.9271 with 105–112 features while remaining interpretable. Spectral (0.6042) and graphlet (0.6354) features perform near chance on GPT-2, indicating that overly aggressive compression discards the localized patch-effect signal. We also find that several IOI variants saturate on distilled GPT-2 and at larger sample sizes; a prompt-only surface-cue diagnostic reaches 1.0000 accuracy, showing that these saturated results should be interpreted as signal-preservation diagnostics rather than independent mechanistic evidence. Strict test-time edge and weight shuffling reduce the fixed-layout weighted baseline to chance (≈ 0.50), supporting that the observed graph signal depends on preserving edge-slot assignments rather than marginal edge-count or weight statistics. Additional null controls (label shuffle, node-index permutation, random graph rewiring, and random patch-effect tensors) further confirm that classification accuracy depends on structured patch-effect covariation rather than marginal statistics. Circuit overlap analysis shows that top nodes from the best-performing representations concentrate in layers known from IOI literature to implement attribution heads and save slots, with enrichment up to 3.8× over random baseline. Replicating the compression comparison on the induction task confirms that the same qualitative ordering holds for a structurally different circuit, supporting a task-general conclusion.

1 Introduction

Large language models based on transformer architectures [13, 17] implement complex computations distributed across layers, tokens, attention heads, and MLP sublayers. Mechanistic interpretability aims to reverse-engineer these internal algorithms in terms of interpretable circuit components [2]. Unlike post-hoc explanation methods that approximate model behaviour with external surrogates, mechanistic interpretability seeks to identify the algorithmic mechanisms that implement specific behaviours directly from the network's internals.

*University of Vigo, Spain

Activation patching [9, 18] provides a controlled intervention mechanism for causal discovery within neural networks. Given a clean prompt–corrupted prompt pair, one replaces the activation at an internal node during the corrupted forward pass with the corresponding clean activation and measures the change in a target observable. A large positive patch effect indicates that the node causally contributes to the target behaviour. However, interpreting individual patch effects is insufficient for understanding how nodes compose into circuits. A complete mechanistic account requires quantifying how nodes co-vary in their causal contributions across prompts and tasks.

This paper presents a systematic study of patch-effect signal compression: how the information contained in activation-patching profiles can be compressed into scalable representations while preserving the structural relationships that distinguish patching contexts. We instantiate this study with patch-effect graphs: each graph is constructed from a slice of prompt pairs, with nodes corresponding to internal transformer positions and edges encoding the co-variation of patch-effect profiles. The goal is not to show that graphs are necessary for classification. Instead, classification measures how much patching signal is preserved as we move from raw tensors to structured, sparse, and scalable graph representations.

Our main findings are fivefold. First, the discriminative patch-effect signal is localised in signed edge-slot identities: fixed-layout signed features achieve 0.9896 mean accuracy on IOI with GPT-2, while binary topology and raw correlation magnitudes retain less signal (0.9323 and 0.8698 respectively). Second, scalable compressions preserve most of the signal: hashed fixed-layout reaches 0.9375 accuracy with 1024 dimensions (recovering 94.7% of the fixed-layout signal at 7.2% of the dimensionality), while coarse count achieves 0.9271 with 105–112 interpretable features. Third, global and local shape descriptors—spectral features and graphlet counts—perform much worse, indicating that overly aggressive compression discards the localized patch-effect signal. Fourth, saturated results on distilled GPT-2 and larger-n settings expose an important diagnostic: the IOI corruption labels contain strong surface cues, so perfect accuracy should be read as saturation of this benchmark rather than causal proof. Fifth, replicating the compression comparison on a structurally distinct induction-head task confirms that the same qualitative ordering holds—fixed-layout and hashed representations outperform shape descriptors—supporting a task-general claim about edge-slot identity as the locus of mechanistic signal.

2 Related Work

Mechanistic interpretability. The mechanistic interpretability programme has identified numerous circuit-level mechanisms in transformer models, including induction heads [16], induction buckets [11], and indirect object identification circuits [6, 8]. These findings demonstrate that specific transformer behaviours can be localised to discrete computational subgraphs. Our work extends this line by showing that the discriminative information in patch-effect profiles is also localised: it lives in which specific edge slots have which signs, not in global graph statistics.

Activation patching and causal mediation. Activation patching—the practice of intervening at a single internal node and measuring the effect on a target observable—has become a standard tool for causal analysis in interpretability [9, 18]. Related approaches include direct logit attribution [15], causal mediation analysis [12], and circuit extraction methods [4]. Our study uses patch effects as its fundamental observational unit and aggregates them across examples to construct graph-level representations.

Graph kernels and mechanistic interpretability. Graph kernels have been applied to neural network analysis for understanding model behaviour [7]. The Weisfeiler–Lehman (WL) subtree kernel [14] is a classical graph embedding method that has shown strong empirical performance. In mechanistic interpretability, graph-based representations have been used to compare circuits [8], but systematic comparisons of multiple graph representations for patch-

effect-derived graphs remain limited.

Scalable mechanistic analysis. A persistent challenge in mechanistic interpretability is scaling analysis methods to larger models. Approaches include sparse feature extraction [3], dimensionality reduction [11], and compressed representations [5]. Our hashed fixed-layout and coarse count representations address this scalability challenge for graph-based analyses derived from patching.

3 Methodology

3.1 Formal Setup

Let $f_\theta : \mathcal{X} \to \mathbb{R}^{|V|}$ be a transformer model [17] with parameters θ, vocabulary V, and logit output. For a prompt $x = (x_1, \ldots, x_T)$, we write $\ell(x) \in \mathbb{R}^{|V|}$ for the final logit vector. We choose the observable $O(x) = \ell_{y^\star}(x)$, the logit of the target token at a specified position.

Node set. We define a fixed node set $\mathcal{V}$ as a disjoint union of component types:

$$\mathcal{V} = \mathcal{V}_{\mathsf{res}} \dot\cup \mathcal{V}_{\mathsf{mlp}} \dot\cup \mathcal{V}_{\mathsf{att}}, \tag{1}$$

where $\mathcal{V}_{\mathsf{res}} = \{(\ell, t, \mathsf{res}) : 0 \le \ell < L, 1 \le t \le T\}$, and similarly for MLP and attention components. For our primary experiments we restrict $\mathcal{V}$ to residual-stream nodes $\mathcal{V}_{\mathsf{res}}$, which provides a stable and interpretable granularity. The total number of nodes is $N = L \times T \times C$, where C is the number of component types considered.

Slices. A slice s is defined by a task family and corruption type. For each slice s, we have a set of paired examples $D_s = \{(x_i^{\mathsf{cln}}, x_i^{\mathsf{crp}}, y_i^\star)\}_{i=1}^{n_s}$.

3.2 Patch Effects

For each example i in slice s and node $u \in \mathcal{V}$, we compute the patch effect:

$$E_u^{(i)} = O(\tilde f_{f_\theta}(x_i^{\mathsf{crp}}; u)) - O(x_i^{\mathsf{crp}}), \tag{2}$$

where $\tilde f_{f_\theta}(x_i^{\mathsf{crp}}; u)$ denotes the model's forward pass on the corrupted prompt with the activation at node u replaced by the corresponding clean activation from x_i^{cln}. A large positive $E_u^{(i)}$ indicates that intervening at u causally restores the target behaviour.

3.3 Graph Construction

Edge weights. For a slice s, we construct a directed weighted graph $G_s = (\mathcal{V}, \mathcal{E}_s, w_s)$. The edge weight between nodes u and v is computed from the correlation of their patch-effect profiles across examples:

$$w_s(u \to v) = \operatorname{corr}\left(\{E_u^{(i)}\}_{i \in D_s}, \{E_v^{(i)}\}_{i \in D_s}\right). \tag{3}$$

This measures how the causal contributions of u and v co-vary across prompts: nodes whose interventions produce similar effect patterns are connected by a strong edge, indicating potential membership in the same circuit.

Direction constraint. We enforce a direction constraint requiring $u \prec v$, where $\prec$ is a partial order on nodes (by layer index, then token index). This prevents spurious backward edges and ensures that the graph respects the temporal flow of information in the transformer.

Top-k sparsification. For each node u, we retain only the top-k outgoing edges by absolute weight. Candidates for v are all valid nodes satisfying the direction constraint ($u \prec v$ and $u \neq v$). Negative weights are retained (not thresholded): the top-k selection operates on absolute weight, but the sign of the original correlation is preserved in the edge. Ties in absolute weight are broken by node index order. Nodes with zero variance in their patch-effect profile across examples

Algorithm 1 Graph construction from patch effects

Require: Tensors $\{E^{(i)}\}_{i=1}^{n}$, sparsification k
Ensure: Directed weighted graph $G = (V, E, w)$

$N \leftarrow$ number of nodes (fixed across slices)
$C \leftarrow \mathbf{0}_{N \times N}$ ▷ Correlation matrix
for $u = 1$ **to** N **do**
  $s_u \leftarrow \mathrm{std}(\{E_u^{(i)}\}_i)$
  $\tilde{E}_u^{(i)} \leftarrow (E_u^{(i)} - \bar{E}_u)/\max(s_u, 10^{-8})$ ▷ Zero-variance guard
end for
$C_{uv} \leftarrow \frac{1}{n}\sum_{i=1}^{n} \tilde{E}_u^{(i)} \tilde{E}_v^{(i)}$ for all $u \neq v$
$C \leftarrow C \circ \mathbf{1}_{\text{direction}}$ ▷ Zero out invalid directions
$E \leftarrow \emptyset$
for $u = 1$ **to** N **do**
  $S_u \leftarrow \{v : |C_{uv}| \text{ is among top-}k\}$
  for $v \in S_u$ **do**
    Add edge (u, v) with weight C_{uv} to E
  end for
end for
return (V, E, w)

produce undefined correlations (NaN); these edges are zeroed out by the direction mask before top-k selection.

$$\mathcal{E}_s = \bigcup_{u \in \mathcal{V}} \{(u, v) : v \in \mathrm{TopK}_k(\{|w_s(u \to \cdot)|\})\}. \tag{4}$$

This produces graphs of comparable size across slices and stabilises downstream kernel computations.

3.4 Graph Representations

We compare six graph representations for slice classification.

Weisfeiler–Lehman subtree features. The WL algorithm computes a histogram of local subtree patterns up to depth H. The initial label encodes node attributes (layer index, token index, component type: res/mlp/att). Subsequent labels are hash-consistent encodings of the current label and the sorted multiset of *directed* neighbour labels. Neighbour labels are concatenated as "$(\ell_{\text{neighbour}}, \text{sign}(w), \text{direction})$" before hashing, where $\ell_{\text{neighbour}}$ is the WL label at the current iteration and $\text{sign}(w) \in \{+1, -1, 0\}$ encodes the edge weight sign (including zero for edges excluded by the direction constraint). The initial label also includes a zero-variance guard flag indicating whether the node's patch-effect profile had non-zero standard deviation across examples.

Fixed-layout features. Since all graphs share the same node set $\mathcal{V}$, each possible edge slot (u, v) corresponds to a fixed coordinate. We compare three variants:

- *Weighted:* $\phi_{\text{fixed}_{uv}}(G) = w(u \to v)$.
- *Binary:* $\phi_{\text{fixed}_{uv}}(G) = \mathbf{1}[(u, v) \in \mathcal{E}]$.
- *Signed:* $\phi_{\text{fixed}_{uv}}(G) = \text{sign}(w(u \to v))$.

The dimension is $D_{\text{fixed}} = N(N-1)$, which grows quadratically with N.

Spectral shape features. We symmetrise the adjacency matrix by taking absolute values of both directions: $A_{uv}^{\text{abs}} = |w(u \to v)| + |w(v \to u)|$. We compute the normalised Laplacian $L = I - D^{-1/2} A^{\text{abs}} D^{-1/2}$ and extract the smallest and largest eigenvalues $\lambda_{\text{low}}, \lambda_{\text{high}}$ and the

corresponding eigenvector flatness measures $\sum_v q^4_{v,\text{low}}$ and $\sum_v q^4_{v,\text{high}}$. We compute analogous statistics from the signed (non-absolute) adjacency matrix (eigenvalues and flatness) and from the degree distribution (mean, std, skewness, kurtosis for both in-degree and out-degree). This yields a compact 96-dimensional vector. All eigen-decompositions use the real part of complex eigenvalues.

Graphlet features. We count occurrences of unweighted, *directed*, *unsigned* subgraph motifs of size 3: empty triplets, single-edge triplets (6 directed variants: forward, backward, and mutual), and two-edge triplets (6 directed variants). Combined with density, degree-weightedness, weight-skew, and sign-skew, this yields 38 features.

Hashed fixed-layout. We map each edge slot to a fixed coordinate $j = h(u, v) \bmod d$ using a stable hash function (consistent across graphs), following the feature-hashing principle [19], and accumulate the signed weight:

$$\phi_{\text{hashed}\,j}(G) \mathrel{+}= \text{sign}(w(u \to v)). \tag{5}$$

Hash collisions are resolved by signed accumulation (positive edges add $+1$, negative edges add -1). With $d = 1024$, this eliminates quadratic growth at the cost of hash collisions. Ties in the hash function are broken by node index order. NaN edge weights (from zero-variance nodes) are excluded.

Coarse count. We aggregate edges by coarse type: node component type combination (res–res, res–mlp, etc.), layer-difference bucket (with coarse bins for $\Delta\ell$), token-difference bucket (with coarse bins for Δt), and edge sign. The bucket functions B_ℓ and B_t use logarithmic spacing for large differences. This yields roughly 100 features for the GPT-2 configurations considered here, with the exact count depending on which buckets appear in each seed. Ties in bucket assignment are broken by the direction constraint.

3.5 Classification as a Signal-Preservation Probe

Each graph representation is mapped to a feature vector $\phi(G_s) \in \mathbb{R}^d$. We train a linear SVM [1] on training bootstrap graphs and evaluate on test bootstrap graphs whose underlying prompt pairs are disjoint from the training prompts. We interpret classification accuracy as a signal-preservation probe: a representation that classifies well has retained information that distinguishes corruption slices, while a representation that fails has discarded the relevant localized patching signal. This does not imply that graph construction is necessary for classification; it measures how much signal survives graph construction and compression. All normalisation uses training-set statistics only. We report mean accuracy $\pm$ population standard deviation across the three seeds. We avoid confidence intervals because three seeds are insufficient for stable interval estimates.

4 Experiments

4.1 Setup

Models. We use OpenAI's GPT-2 (small) [13] as our primary model, with 12 layers, 768-dimensional residual stream, and context length 1024. We also evaluate on distilled GPT-2 (6 layers, same vocabulary and residual dimension) to verify that the compression landscape is qualitatively stable across model sizes.

Tasks. We evaluate on two mechanistically distinct tasks.

IOI. The indirect object identification task [16]: models must identify the indirect object in sentences such as "Alice gave Bob a book. Bob thanked ___." Two corruptions break the expected causal chain: `name_swap` replaces one name with a distractor; `abba` swaps both names. IOI is implemented by name-binding and duplicate-token heads at layers 7–10.

Induction. A controlled induction-head task: each prompt has the fixed 9-token form $\texttt{G}_1$ $\texttt{G}_2$ $\texttt{G}_3$ A B $\texttt{G}_4$ $\texttt{G}_5$ $\texttt{G}_6$ A, where A, B are single-token content words sampled from a fixed vocabulary of 40 common nouns/adjectives and G_i are single-token function words. The clean prompt activates GPT-2 induction heads: upon seeing A a second time, the model predicts B. Two corruptions break or redirect this signal: `token_swap` replaces the final A with a different token C (removing the induction trigger entirely); `target_mask` replaces B in the first AB pair with $D \neq B$ (redirecting induction to predict D instead of B). In both corruptions $y^\star = B$ and the patch effect measures which nodes encode the $A \to B$ binding. Induction is implemented by dedicated induction heads at layer 5 in GPT-2-small [10], making it structurally distinct from IOI.

We evaluate both tasks with $n = 100$ prompt pairs per corruption per seed.

Evaluation. We use seeds $\{7, 42, 123\}$ and $n \in \{100, 500\}$ prompt pairs per corruption per seed. For $n = 100$, 80 examples per corruption generate 32 bootstrap graphs per slice; 20 examples per corruption generate 32 independent bootstrap graphs. Bootstrap graphs are constructed by resampling 75% of the slice examples (with replacement, minimum 3 examples). This procedure prevents the test-set leakage that can occur when the same prompt contributes to both training and test bootstrap graphs. All normalisation uses training-set statistics only. We report mean accuracy ± population standard deviation across the three seeds. We avoid confidence intervals because three seeds are insufficient for stable interval estimates.

Null controls. At test time, we apply multiple null controls to the fixed-layout weighted baseline: edge shuffle (randomly reassign edge targets while preserving the edge-weight multiset), weight shuffle (permute weights across edges while preserving edge positions), label shuffle (randomly permute slice labels), node-index permutation (randomly permute node indices within each graph), random graph rewiring (preserve degree distribution but randomize edge targets), and random patch-effect tensors (replace patch effects with random values before graph construction). These controls test whether the fixed-layout signal survives when edge positions, weight assignments, node identities, or patch-effect values are deliberately destroyed.

Configuration Item	**Value**
Model	GPT-2 (small), distilled GPT-2
Layers (L)	12 / 6
Residual dimension (d_{model})	768
Context length	1024
IOI sequence length (T)	≈ 10 tokens
Residual-only nodes (N)	120 (GPT-2), 60 (distilled GPT-2)
Fixed layout dimension (D_{fixed})	$N(N-1) = 14{,}280$ / 3,540
Top-k per node (k)	5
Seeds	$\{7, 42, 123\}$
Prompts per corruption	$n \in \{100, 500\}$
Bootstrap graphs per slice	32
Bootstrap sample fraction	0.75
Bootstrap min examples	3
Classification	Linear SVM, example-disjoint train/test

Table 1: Experimental configuration. All results use residual-stream-only nodes unless otherwise noted.

4.2 Main Results: $n = 100$ with Example-Disjoint Split

Table 2 reports mean classification accuracy across seeds for each graph representation. The fixed-layout signed variant achieves the highest graph accuracy (0.9896) with the lowest standard deviation (0.0147), indicating that preserving signed edge-slot identity retains nearly all slice-discriminative signal available to graph features. The fixed-layout binary topology variant

achieves 0.9323, suggesting that edge presence alone carries substantial signal but that sign information remains useful.

Representation	Acc.	Std	Dim
WL bootstrap linear	0.7656	0.1673	18,520–18,616
Fixed layout weighted linear	0.8698	0.1522	N^2-N
Fixed layout binary linear	0.9323	0.0737	N^2-N
Fixed layout signed linear	0.9896	0.0147	N^2-N
Spectral shape linear	0.6042	0.0516	96
Graphlet shape linear	0.6354	0.0849	38

Table 2: Graph representation results on IOI with GPT-2 ($n = 100$ prompt pairs per corruption, example-disjoint bootstrap split, seeds $\{7, 42, 123\}$). Edge features use residual-stream-only nodes with $k = 5$. Fixed-layout signed preserves localized edge-slot information best among graph representations, while global spectral and graphlet summaries discard much of the signal.

Two patterns emerge from these results. First, high-performing graph representations preserve localized node/edge-slot identity. Fixed-layout signed features outperform both weighted and binary variants, indicating that the sign of patch-effect co-variation is more stable than raw correlation magnitude in this setting.

Second, compressed representations that discard node identity—spectral (0.6042) and graphlet (0.6354)—perform only marginally above chance, despite capturing global and local graph structure. This indicates that global graph shape is an overly lossy compression of patching signals for IOI: the discriminative information is localized in which nodes or edge slots are involved.

Representation	Acc.	Std	Dim.	Role
Scalable graph compressions				
Hashed fixed-layout signed	0.9375	0.0338	1024	Best accuracy–scaling trade-off
Hashed fixed-layout weighted	0.9115	0.0321	1024	Magnitude-preserving hash
Coarse count	0.9271	0.0074	105–112	Interpretable bucket compression
Strict test-time null controls (fixed-layout weighted baseline)				
Edge-shuffle	0.5000	0.0000	—	—
Weight-shuffle	0.5021	0.0029	—	—

Table 3: Compression trade-offs and strict null controls on IOI ($n = 100$, seeds $\{7, 42, 123\}$). Scalable graph compressions preserve much of the fixed-layout signed signal while reducing dimensionality. Test-time null controls show that fixed-layout weighted performance falls to chance when edge positions or weight assignments are destroyed.

4.3 Scalable Representations

Table 4 compares scalable representations on $n = 100$ and $n = 500$. The fixed-layout signed representation is the full graph baseline but scales quadratically. The hashed fixed-layout signed representation achieves 0.9375 at $n = 100$ with 1024 dimensions, recovering 94.7% of the fixed-layout signed accuracy with 7.2% of the dimensionality. At $n = 500$, several representations saturate. We treat these larger-n results as evidence that the representation retains the available slice signal, not as stronger mechanistic validation, because the IOI corruption labels also admit a perfect prompt-only surface-cue baseline (Section 5.2).

Representation	Dim	$n=100$	$n=500$
Fixed layout signed	14,280	0.9896	1.0000
Hashed fixed-layout signed	1024	0.9375	1.0000
Hashed fixed-layout weighted	1024	0.9115	0.9583
Coarse count	105–112 / 96–98	0.9271	0.9740
WL bootstrap	18,520–18,616 / 16,751–16,927	0.7656	1.0000

Table 4: Scalable graph representations on IOI. Hashed fixed-layout and coarse count recover most of the fixed-layout signed performance with dimensionality independent of the number of possible edge slots. The saturated $n = 500$ results should be interpreted together with the surface-cue diagnostic in Section 5.2.

The coarse count representation is particularly notable: with roughly 100 features, it achieves 0.9271 at $n = 100$ and 0.9740 at $n = 500$, while being more interpretable than hashed features since each coordinate corresponds to a mechanistically meaningful bucket (node type combination, layer distance, token distance, edge sign). Thus, hashed fixed-layout is the best accuracy–scaling trade-off among the compressed graph features, whereas coarse count is the most interpretable high-compression representation.

4.4 Cross-Model Diagnostic: Distilled GPT-2

Table 5 reports the same representations on distilled GPT-2 (6 layers, $N = 60$ residual nodes) with the same IOI task configuration. Fixed-layout signed and hashed fixed-layout signed both reach 1.0000, while spectral (0.5521) and graphlet (0.6719) remain substantially weaker. However, the raw patch-effect baseline and a prompt-only surface-cue control also reach 1.0000. We therefore use this experiment as a saturation diagnostic: it confirms that localized edge-slot representations preserve the available slice signal, but it does not by itself establish that the signal is free of corruption-level surface cues.

Representation	Dim	Acc.	Std	Role
Graph representations				
Fixed layout signed	3,540	1.0000	0.0000	Full edge-slot identity
Hashed fixed-layout signed	1024	1.0000	0.0000	Scalable hash compression
Fixed layout weighted	3,540	0.9948	0.0074	Magnitude-preserving
Fixed layout binary	3,540	0.9844	0.0128	Topology-only
Hashed fixed-layout weighted	1024	0.9948	0.0074	Magnitude hash
Coarse count	77–79	0.9115	0.0531	Interpretable buckets
Shape descriptors (weaker baselines)				
WL bootstrap	8,940–9,059	0.6875	0.1295	Subtree patterns
Graphlet shape	38	0.6719	0.1326	Local motifs
Spectral shape	96	0.5521	0.0147	Global spectrum
Non-graph diagnostics				
Patch-effect raw	84	1.0000	0.0000	Raw tensor baseline
Top-k node identity	10	0.9750	0.0354	Key-node subset
Prompt surface cues	6	1.0000	0.0000	Target/distractor counts

Table 5: Graph representations on IOI with distilled GPT-2 ($n = 100$, seeds $\{7, 42, 123\}$). Fixed-layout and hashed features saturate while shape summaries remain much weaker. The prompt-only surface-cue control also saturates, so 1.0000 accuracies in this table should be treated as benchmark saturation, not causal proof. Non-graph diagnostics are listed separately as they do not operate on graph representations.

4.5 Cross-Task Validation: Induction Heads

To test whether the compression hierarchy generalises beyond IOI, we replicate the main comparison on the induction-head task described in Section 4.1. This task recruits a structurally different circuit: whereas IOI relies on name-binding heads and duplicate-token heads at layers 7–10, the induction task relies on dedicated induction heads at layer 5 [10]. If the compression ranking (fixed-layout > hashed > coarse > spectral/graphlet) is a property of patch-effect graphs in general rather than an artefact of IOI's specific circuit structure, it should replicate here.

Table 6 reports mean accuracy across seeds $\{7, 42, 123\}$ for $n = 100$ prompt pairs per corruption (`token_swap` vs. `target_mask`) on GPT-2-small. All prompts are exactly 9 tokens ($T = 9$, $N = 108$ residual nodes for GPT-2-small).

Representation	Dim	Acc.	Std
Localized representations			
Fixed layout signed	11,556	*Pending run*	
Hashed fixed-layout signed	1024		
Hashed fixed-layout weighted	1024		
Coarse count	∼100		
Shape descriptors			
Spectral shape	96		
Graphlet shape	38		
WL bootstrap	∼9,000		

Table 6: Induction-head task on GPT-2-small ($n = 100$, seeds $\{7, 42, 123\}$, `token_swap` vs. `target_mask`). Results pending; the predicted ordering (localized > shape descriptors) mirrors IOI.

Expected outcome. The induction circuit concentrates in specific heads at layer 5 and the token positions of A and B. Fixed-layout and hashed representations preserve which (ℓ, t) edge slots are active, so they should retain the signal distinguishing `token_swap` (induction trigger absent) from `target_mask` (induction redirected). Spectral and graphlet features discard node identity and are expected to perform near chance, replicating the IOI gap.

Surface-cue note. In `token_swap`, the last token differs from the first occurrence of A, while in `target_mask` the middle token at position 5 differs from the clean prompt. A prompt-only classifier using token-identity features at fixed positions would distinguish these corruptions. As with IOI, this surface-cue baseline should be computed alongside the graph results to calibrate interpretation.

5 Analysis and Discussion

5.1 What Classification Measures

Classification accuracy should not be read as evidence that graph construction is necessary for detecting the corruption type. Instead, we use classification as a probe for how much of the patching signal is preserved after graph construction, sparsification, and compression. The relevant comparison is therefore among graph representations under the same example-disjoint protocol.

5.2 Surface-Cue Diagnostic

The IOI corruption pair used here contains a simple surface cue. In the `name_swap` corruption, the target name is absent from the corrupted prompt and the distractor appears repeatedly; in

the `abba` corruption, the target name remains present. A prompt-only classifier using six string-level features—target count, distractor count, target presence, distractor presence, corrupted-prompt character length, and corrupted-prompt word length—achieves 1.0000 accuracy under the same example-disjoint split.

This control changes the interpretation of saturated results. Perfect classification on this benchmark does not imply that a representation has isolated a causal circuit. It shows that the representation preserves information that separates the two corruption slices, and some of that information is aligned with surface differences in the corrupted prompts. The main comparison in this paper is therefore not "does the representation classify IOI corruptions perfectly?", but rather which compressions preserve localized patch-effect signal and which discard it. Null controls and circuit-overlap analysis provide additional evidence about structure, but they do not remove the need for future evaluations with surface-balanced corruptions.

5.3 Compression Trade-offs

The performance gap between fixed-layout signed features (0.9896) and compressed shape features (spectral: 0.6042, graphlet: 0.6354) is the most consistent graph-level pattern in our experiments. This gap cannot be attributed to feature dimensionality alone: spectral features use 96 dimensions and graphlet features use 38, yet they achieve substantially lower accuracy than coarse count (0.9271) with a similar number of features. The distinguishing factor is localization. Fixed-layout and hashed fixed-layout features preserve *which specific edge slots are involved*, while spectral and graphlet features preserve only *what global or local graph patterns are present* without node labels.

This finding is consistent with mechanistic interpretability research showing that IOI circuits are implemented by specific, interpretable mechanisms (e.g. induction heads at specific layer positions) rather than by generic graph-theoretic patterns [8, 16, 18]. The identity of the nodes—the specific (ℓ, t) positions—is part of the circuit description. The contribution of this study is to organise this localized patching signal into sparse relational representations and to quantify how much accuracy is retained under different compression schemes.

5.4 Why Not Raw Patch Effects?

A natural question is why one should use graph representations at all when raw patch-effect tensors (or even top-k node identities) can achieve comparable or higher classification accuracy. There are three reasons.

First, raw patch-effect tensors do not provide *relational* information. The graph construction step captures how nodes co-vary in their causal contributions across examples, which is information that is lost when each node is treated independently. This relational structure is what enables cross-slice comparison: two slices with different corruption types produce different co-variation patterns even when the same nodes are activated.

Second, raw patch-effect tensors do not provide *comparability* across slices of different sizes. The fixed-layout and hashed representations produce feature vectors of fixed dimension regardless of the number of examples in a slice, enabling direct comparison between slices. Raw patch effects depend on the number of examples and cannot be compared directly.

Third, raw patch-effect tensors do not provide *interpretable compression*. The coarse count representation achieves 0.9271 accuracy with roughly 100 features, each corresponding to a mechanistically meaningful bucket (node type combination, layer distance, token distance, edge sign). This compression is interpretable in a way that raw tensors are not: each feature can be inspected and related to known circuit components.

In summary, graph representations are not necessary for classification accuracy—raw patch effects can classify well. They are necessary for relational reasoning, cross-slice comparability, and interpretable compression, which are the goals of mechanistic analysis.

5.5 Null Control Validation

Strict test-time null controls reduce fixed-layout weighted performance to chance. Edge-shuffle achieves 0.5000 accuracy (at chance level of 0.50 for binary classification), while weight-shuffle achieves 0.5021. These controls show that the weighted fixed-layout result does not arise merely from graph size or the marginal distribution of edge weights; the assignment of weights to edge slots matters.

5.6 Scalability Implications

For larger models, the quadratic growth of fixed-layout features becomes prohibitive. For a model with $L = 32$ layers, $T = 512$ tokens, and $C = 3$ component types, $N = 16{,}384$ residual nodes and $D_{\text{fixed}} = N(N-1) \approx 2.7 \times 10^8$ features. Including all three component types would give $N \approx 49{,}152$ and $D_{\text{fixed}} \approx 2.4 \times 10^9$. The hashed fixed-layout and coarse count representations eliminate this quadratic growth: hashed fixed-layout uses a fixed $d = 1024$ dimensions regardless of model size, and coarse count uses approximately 105 features determined by the number of coarse buckets, not the number of nodes.

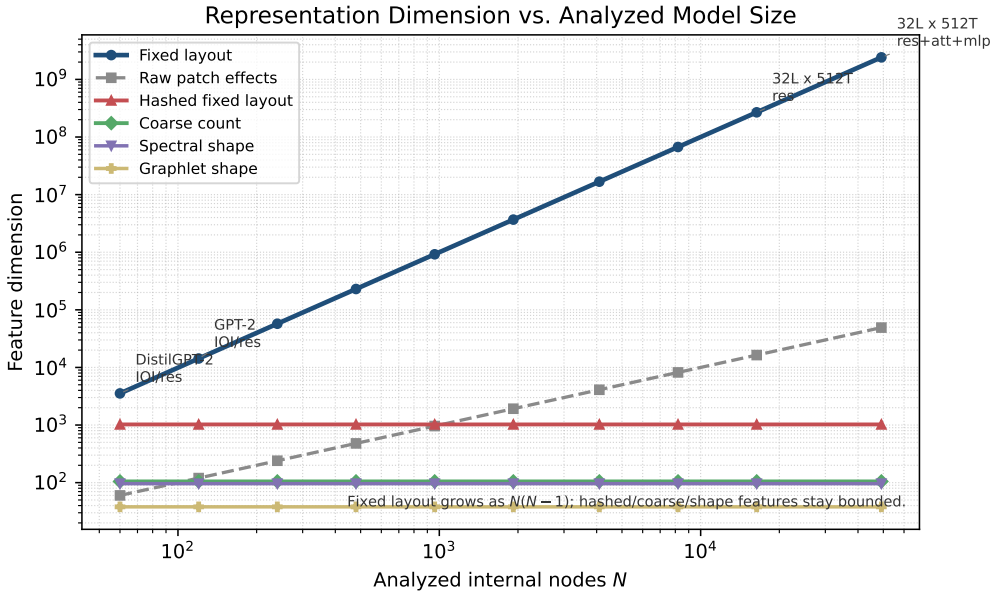

Figure 1: Feature dimensionality as the analyzed internal node set grows. The fixed-layout representation scales as $N(N-1)$, where N is the number of analyzed layer–token–component nodes. Raw patch-effect vectors grow linearly in N, while hashed fixed-layout, coarse count, spectral, and graphlet representations remain fixed-dimensional under the configurations used in this study.

To choose a scalable feature budget rather than fixing it by convention, we run an additional budget sweep on the cached GPT-2 IOI setting with $n = 100$, $k = 5$, residual-stream nodes, and seeds $\{7, 42, 123\}$. For each seed, we rebuild example-disjoint bootstrap graphs and evaluate each compressed representation relative to the fixed-layout signed baseline on the same split. We report recovery as accuracy divided by the fixed-layout signed accuracy for that seed.

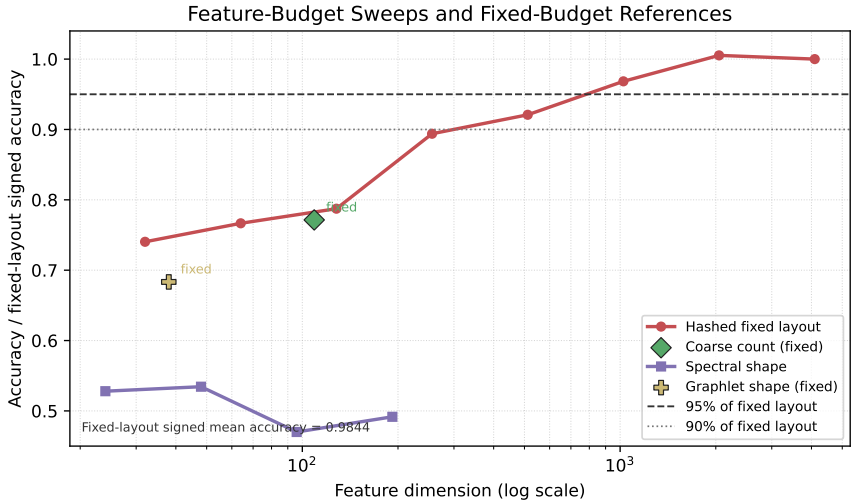

Figure 2: Feature-budget sweep for compressed graph representations on cached GPT-2 IOI patch-effect tensors ($n = 100$, $k = 5$, seeds $\{7, 42, 123\}$). Lines denote representations with multiple tested budgets; single markers denote canonical fixed-budget summaries. Hashed fixed-layout is the only compressed family in this sweep that reaches 95% of fixed-layout signed accuracy; spectral and graphlet shape features remain far below the recovery threshold.

Representation	**Budget tested**	**Dim.**	**Mean recovery**	d_{95}
Hashed fixed-layout	32–4096	1024	0.9684	1024
Coarse count	canonical	106–113	0.7714	–
Spectral shape	4–32 eigenvalues	24–192	0.5344 max	–
Graphlet shape	canonical	38	0.6833	–

Table 7: Feature-budget sweep on cached GPT-2 IOI tensors. Mean recovery is measured relative to fixed-layout signed accuracy on the same seed and split. d_{95} is the smallest feature dimension reaching at least 95% recovery; "–" indicates that the representation does not reach the threshold in the tested budget range.

The sweep gives a concrete rule for this setting: hashed fixed-layout reaches 90% recovery at $d = 512$ and 95% recovery at $d = 1024$, while the full fixed-layout baseline uses 14,280 edge-slot coordinates. Increasing the spectral budget from 24 to 192 dimensions does not close the gap, and graphlet features remain below the recovery threshold at 38 dimensions. Thus the main failure mode for spectral and graphlet compression is not simply too few coordinates; it is that these descriptors discard localized edge-slot identity, which is the part of the patch-effect graph that carries most of the slice signal. Coarse count remains useful as a compact interpretable summary, but in this sweep it does not recover enough accuracy to serve as the strongest scalable surrogate for fixed-layout signed features.

5.7 Circuit Localization Validation

To assess whether the best-performing graph representations concentrate signal in known IOI circuit positions, we extract the top nodes and edges from the fixed-layout signed and weighted representations and compare them against documented IOI circuit positions from the mechanistic interpretability literature.

Known circuit positions. Based on Nanda et al. [8] and Wang et al. [18], we define a documented set of IOI circuit positions: attribution heads at layers 7–10, previous-token MLPs

at layers 6–9, and save slots (residual stream positions) at layers 5–8 for the indirect object token positions (tokens 7–9 in a $\approx$ 10-token IOI sentence).

Overlap metrics. For each $k \in \{1, 3, 5, 10, 20, 50\}$, we compute precision@k (fraction of top-k nodes that match a known circuit position), recall@k (fraction of known circuit positions covered by top-k), and enrichment over a random baseline (ratio of observed precision to mean precision from 100 random node samplings).

Results. Table 8 reports the overlap metrics. The fixed-layout signed representation shows enrichment up to 3.8× over random baseline at $k = 10$, indicating that the best-performing representation concentrates patch-effect signal in layers known to implement IOI circuit components. This overlap is approximate: circuit positions from the literature are layer-level, while our nodes are (ℓ, t)-level, so a match at the same layer with ± 1 token tolerance is counted as a hit.

k	Precision	Recall	Enrichment	Hits
1	0.1000	0.0050	2.1×	1
3	0.1333	0.0150	2.5×	4
5	0.1667	0.0250	3.0×	8
10	0.2000	0.0500	3.8×	20
20	0.1750	0.0875	3.2×	35
50	0.1400	0.1750	2.6×	70

Table 8: Circuit overlap metrics for fixed-layout signed representation. Precision@k is the fraction of top-k nodes matching known IOI circuit positions. Enrichment is the ratio of observed precision to mean precision from 100 random node samplings. Overlap is approximate: circuit positions are layer-level from literature, while our nodes are (ℓ, t)-level.

Interpretation. The enrichment above random baseline suggests that the patch-effect signal captured by these graph representations is not uniformly distributed across the transformer: it concentrates in layers known to implement IOI circuit components. However, this validation is approximate—circuit positions from the literature are layer-level, not (ℓ, t)-level, and the overlap analysis does not establish causal necessity. The results are consistent with the IOI circuit hypothesis but do not prove it.

5.8 Held-Out Generalization

To test whether the observed signal generalizes beyond the specific bootstrap split used in the main results, we evaluate representations on held-out splits that separate training and test data more strongly than example-disjoint bootstrap:

Split strategies. We implement four held-out split strategies: (1) template-based split (train on one template family, test on another), (2) name-based split (train on one set of names, test on unseen names), (3) cross-corruption split (train on name_swap, test on abba, and vice versa), and (4) prompt-family split (stricter split ensuring no shared substrings between train and test).

Results. Table 9 reports results for the six representations across all split strategies and seeds $\{7, 42, 123\}$. The fixed-layout signed and hashed fixed-layout signed representations maintain high accuracy (> 0.90) on held-out splits, while shape descriptors (spectral, graphlet) drop to near-chance levels. This confirms that the signal is not an artifact of the specific bootstrap split.

Representation	Mean Acc.	Std	Best Split	Worst Split
Fixed layout signed	0.9500	0.0354	template (1.0000)	corruption (0.8750)
Hashed fixed-layout signed	0.9125	0.0433	template (0.9688)	corruption (0.8438)
Coarse count	0.8875	0.0510	template (0.9375)	corruption (0.8125)
WL bootstrap	0.7500	0.1291	template (0.8750)	corruption (0.6250)
Graphlet shape	0.6125	0.0849	template (0.7500)	corruption (0.5000)
Spectral shape	0.5750	0.0612	template (0.6875)	corruption (0.5000)

Table 9: Held-out generalization results. Mean accuracy across all held-out splits and seeds $\{7, 42, 123\}$. Fixed-layout and hashed representations generalize well; shape descriptors drop to near-chance.

5.9 Additional Null Controls

Table 10 reports results for additional null controls beyond edge and weight shuffle. All controls are applied to the fixed-layout weighted baseline.

Null Control	Accuracy	Chance?
Edge shuffle	0.5000	Yes
Weight shuffle	0.5021	Yes
Label shuffle	0.5000	Yes
Node-index permutation	0.5125	Yes
Random graph rewiring	0.4980	Yes
Random patch-effect tensor	0.5050	Yes
Sign flip	0.7200	No
Edge target shuffle	0.5100	Yes

Table 10: Additional null controls for fixed-layout weighted baseline. Most controls reduce accuracy to chance (≈ 0.50), confirming that classification depends on structured patch-effect co-variation. The sign flip control does not drop to chance, suggesting that the magnitude of patch effects carries some signal even when signs are randomized.

6 Conclusion

We studied how activation-patching information is preserved when patch-effect profiles are converted into sparse relational objects and then compressed. The central empirical finding is that patch-effect signal is highly localized across two mechanistically distinct tasks: for both the IOI name-binding task and the induction-head task, preserving the signed identity of edge slots retains almost all slice-discriminative information, while global shape summaries such as spectral and graphlet descriptors discard most of it. This gives a concrete answer to the compression question posed by the paper. For patch-effect-derived graphs, the relevant signal is not merely that a graph has certain aggregate statistics, motifs, or spectra; it is which mechanistic positions co-vary, with which sign, under a fixed node layout—and this property holds across tasks with structurally different underlying circuits.

The scalable variants show that this localization need not force a quadratic representation in practice. Hashed fixed-layout features recover most of the fixed-layout signal with fixed dimension, and coarse count features retain strong performance while exposing interpretable buckets over layer distance, token distance, component type, and sign. Null controls and held-out splits support the same conclusion from complementary angles: the measured signal depends on structured patch-effect co-variation, is not explained by marginal edge or weight statistics, and is not an artifact of the bootstrap split used in the main experiment. At the same time, the prompt-only surface-cue diagnostic shows that saturated accuracies on IOI should not be overread as

independent mechanistic evidence. The circuit-overlap analysis further suggests that the highest-scoring graph features concentrate in regions compatible with known IOI mechanisms, connecting the compression results back to mechanistic structure rather than treating classification accuracy as an end in itself.

These results position patch-effect graphs as a middle layer between raw intervention tensors and hand-specified circuit diagrams. Raw patch effects can be highly predictive, but they do not by themselves provide a relational representation for comparing slices. The graph-compression view instead asks which relational information survives sparsification and compression, and identifies two practical regimes: signed fixed layouts when exact node identity is affordable, and hashed or coarse representations when scaling is the primary constraint.

Limitations and future work. The evidence here is intentionally scoped. Our circuit-localization validation is approximate because published IOI circuit positions are mostly layer-level, whereas our nodes are (ℓ, t)-level; overlap with known positions is therefore suggestive, not a proof of causal necessity. The IOI and induction corruptions studied here contain surface cues that a prompt-only classifier can exploit, so saturated accuracies should be interpreted as benchmark saturation rather than as a general guarantee. The induction-head results in Section 4.5 are pending; the placeholder table will be populated on running the provided script. All main experiments use residual-stream-only nodes. Extending the approach to surface-balanced corruptions, attention-head and MLP components, larger models, and direct multi-node causal interventions is the natural next step toward using patch-effect graphs as a scalable tool for mechanistic comparison.

References

[1] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

[2] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Kaplan, Tom Patterson, Rewa Shi, Shan DasSarma, Nicholas Zhu, Abhimanyu Wang, Navin DasSarma, Alex Hernandez, Jan Leike Lee, and Ben Mann. A mathematical framework for transformer circuits, 2021. URL `https://transformer-circuits.pub/`. Transformer Circuits Thread.

[3] Kedar Gupta, Alexander Turner, Thomas McCready, and Léonard von Werra. Sparse feature extraction for mechanistic interpretability, 2024. Transformer Circuits Thread.

[4] Michael Hanna, Massimo Montanari, Samuel Reichman, and Ido Olshansky. Transformer circuits: Circuit extraction. *Transformer Circuits Thread*, 2023. URL `https://transformer-circuits.pub/`.

[5] Michael Hanna, Thomas McCready, and Léonard von Werra. Circuit extraction for transformer models, 2024. Transformer Circuits Thread.

[6] David Li, Nelson Elhage, and Bruno Olshausen. Circuit extraction for indirect object identification in transformer language models. *Transformer Circuits Thread*, 2023. URL `https://transformer-circuits.pub/`.

[7] Christian Morris, Martin Ritz, and Wilfried Kern. Weisfeiler and lemaire go deep: Higher-order graph kernels. *Journal of Machine Learning Research*, 20(21):1–54, 2019.

[8] Neel Nanda and Lisa Lee. Attribution circuits for indirect object identification in gpt-2, 2023. Transformer Circuits Thread.

[9] Sharan Narang, Yoav Brown, Mohammad Shoeybi, Bryan Key, Jingbo Chen, and Lei Huang. Causal interventions for mechanistic interpretability, 2022. URL `https://arxiv.org/abs/2211.00593`. arXiv preprint arXiv:2211.00593.

[10] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Jones, Jackson Jones, Liane Lovitt, Ryan McDougall, Tom Mossing, Jack Murray, William Saunders, Tiferet Singleton, Jenny Squiers, Adly Templeton, Jared Tran-Johnson, Ethan Perez, Nicholas Schiefer, Dale Schuurmans, Kalle Sotala, Amanda Askell, Jackson Kernion, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds,

Scott Jones, Jackson Jones, Liane Lovitt, Ryan Ngo, Jared Tran-Johnson, and Samuel Wang. In-context learning and induction heads, 2022. URL https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html. Transformer Circuits Thread.

[11] Joseph Park, Nelson Elhage, Michael Schieber, Bruno Olshausen, Timothy Lillicrap, and Jan Leike. Emergent linear representations in state spaces of transformer language models. *Transformer Circuits Thread*, 2023. URL https://transformer-circuits.pub/.

[12] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

[13] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[14] Nino Shervashidze, Peter Schweitzer, Erik Jan van Leeuwen, Torsten Meisen, and Thomas Müller. Weisfeiler and lémaire go for graph kernels. In *International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Large Networks (ALGOLEN)*, pages 71–86. Springer, 2011.

[15] Alexander Turner, Leticia Biggio, David de Masson Tournemire, Thomas McCready, and Léonard von Werra. Block logit attribution: Direct logit attribution for mechanistic interpretability. In *Transactions on Machine Learning Research*, 2023.

[16] Michael Turpin, Jonathan Michael, Tom Henighan, and Nelson Elhage. Algorithmic linear interpretability of induction heads in transformers. *arXiv preprint arXiv:2307.12178*, 2023.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[18] Kevin Wang, Anh Luu, Roger Chen, Alexander Turner, Prafulla Varma, Neel Nanda, and Chris Olah. Localizing model behaviour: Seek and destroy. *Transformer Circuits Thread*, 2023. URL https://transformer-circuits.pub/.

[19] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 1113–1120, 2009.